IF2110 Algoritma dan Pemrograman 2

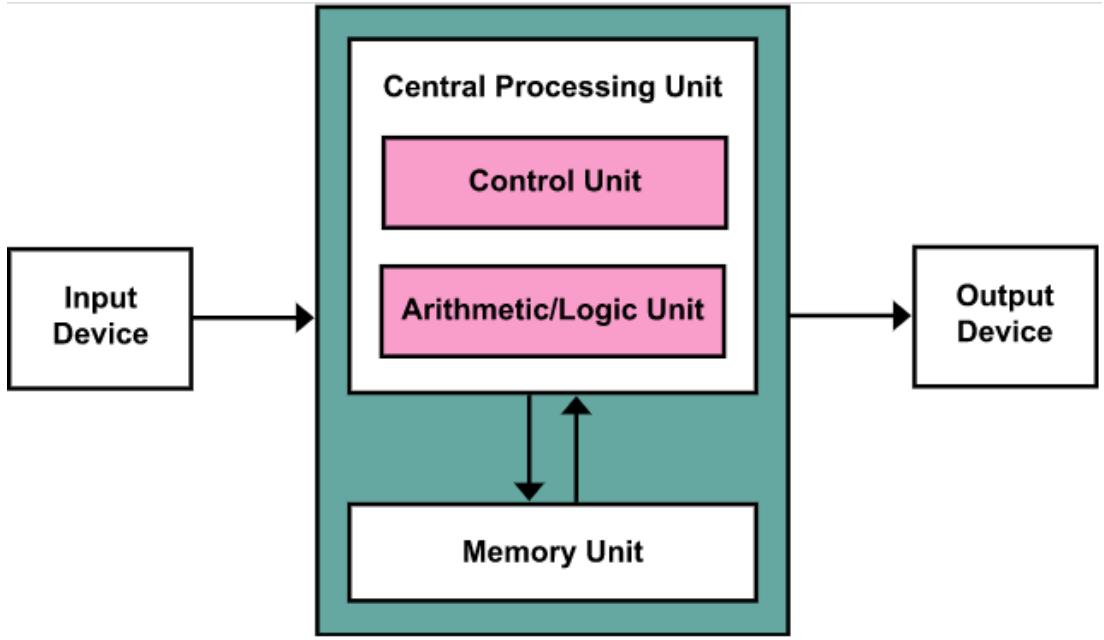
Review Pemrograman Prosedural

Tim Pengajar IF2110

Sekolah Teknik Elektro dan Informatika

Paradigma Imperatif/Prosedural

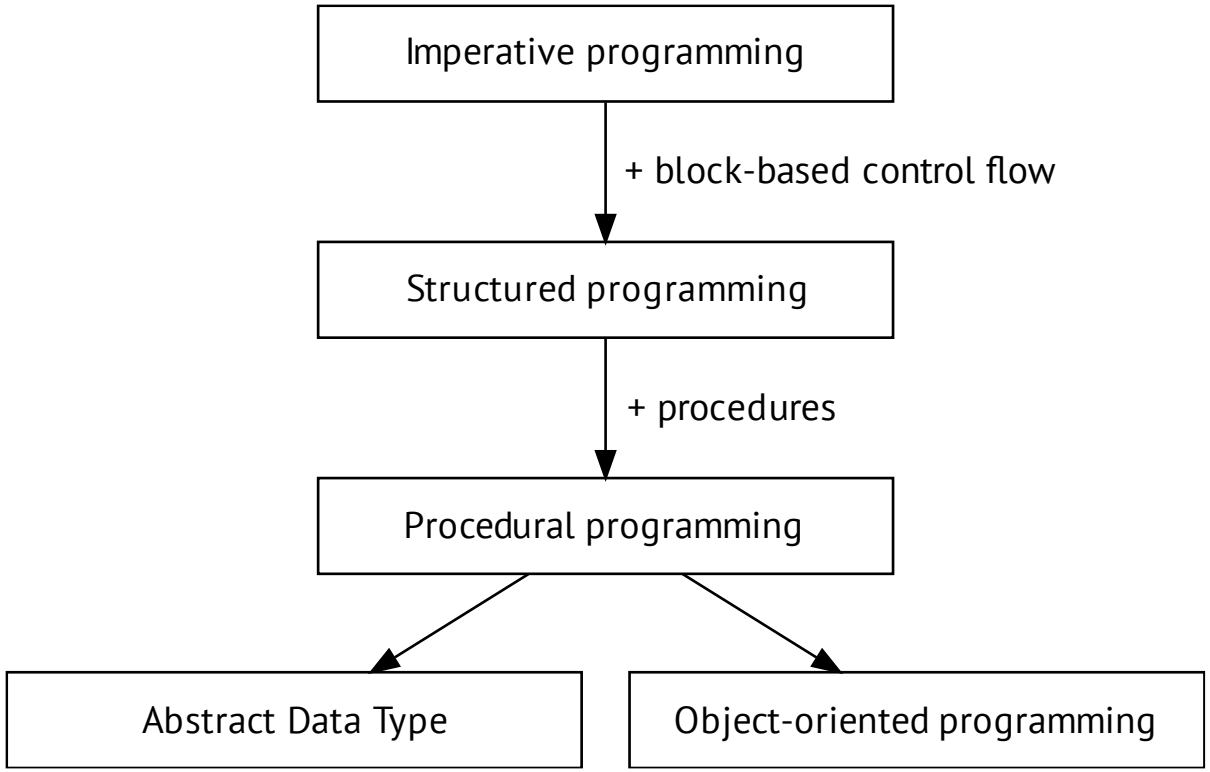
- Didasari oleh konsep mesin Von Neumann (*stored program concept*)
- Program didasari oleh **strukturisasi informasi** di dalam memori dan **manipulasi** dari informasi yang disimpan tersebut



Skema arsitektur mesin Von Neumann

Program = Algoritma + Struktur Data

Sejarah paradigma prosedural



Struktur Dasar Program Prosedural - Notasi Algoritmik

Program <JudulProgram> { Spesifikasi Program }
KAMUS { Deklarasi type, variabel, konstanta, fungsi, prosedur }
ALGORITMA { Deretan langkah algoritmik untuk penyelesaian persoalan }

Struktur Dasar Program – Bahasa C

```
{ Program <JudulProgram> }
{ Spesifikasi Program }

int main () {
    { KAMUS }
    { Deklarasi variabel, konstanta, fungsi, prosedur }

    { ALGORITMA }
    { Deretan langkah algoritmik untuk penyelesaian persoalan }
    return 0;
}
```

Skema standar

- Skema standar adalah skema umum yang sering digunakan untuk memecahkan kasus-kasus trivial (contoh: pemasukan nilai, penampilan nilai, pencarian nilai, pengurutan, dll)
- Skema standar yang telah dipelajari di IF1210:
  - Skema proses validasi (I dan II)
  - Skema pengulangan
  - Skema pemrosesan sekuensial: skema tanpa mark, skema dengan mark
  - Skema pemrosesan array: skema traversal, skema pencarian nilai ekstrim, skema searching, skema sorting
  - Skema pemrosesan file I/O

Apa pentingnya skema standar?

- Penggunaan pola yang seragam antar programmer
- Mempermudah pembacaan dan pemeriksaan kode sumber oleh orang lain
- Mempercepat penulisan program untuk kasus-kasus yang umum (pengisian, penulisan, validasi, pencarian nilai ekstrim, dll)

Beberapa contoh skema yang telah dipelajari

Skema Proses Validasi

```
SKEMA PROSES_VALIDASI
{ Skema program yang menerima input data,
  hanya melakukan proses jika data valid }
KAMUS
{ deklarasi data input }
ALGORITMA
   input(<data>)
   if (<data_valid>) then
     { <data_valid> adalah predikat, suatu ekspresi
        boolean yang menyatakan validitas dari data }
     { Lakukan_proses }
   else { data tidak valid }
      output(<pesan_kesalahan>)
```

Jenis-Jenis Skema Pengulangan

1. Berdasarkan jumlah pengulangan
2. Berdasarkan kondisi berhenti
3. Berdasarkan kondisi mengulang
4. Berdasarkan dua aksi
5. Berdasarkan pencacah

Skema Pemrosesan Sekuensial pada Array

- Merupakan **pemrosesan sekuensial tanpa mark**
- Dimungkinkan adanya akses langsung jika indeks terdefinisi
  - First-Elmt adalah elemen tabel dengan indeks terkecil
  - Next-Elmt dicapai melalui suksesor indeks
- Model akses sekuensial tanpa mark
  - kondisi berhenti adalah jika indeks sudah mencapai harga indeks yang terbesar yang telah terdefinisi
- Tabel tidak mungkin “kosong”
  - jika kita mendefinisikan tabel, maka minimal mengandung sebuah elemen

Algoritma Pencarian Sequential Skema Pencarian Tanpa Boolean

```
procedure SEQSearchX1 (input T : TabInt, input N : integer,
                       input X : integer, output IX : integer)
{ Mencari harga X dalam Tabel T [1..N] secara sekuensial mulai
  dari T1. Hasilnya adalah indeks IX di mana TIX = X (i
  terkecil), IX = 0 jika tidak ketemu }
```

```
Kamus Lokal
    i : integer { [1..NMax], indeks untuk pencarian }
```

```
ALGORITMA
    i ← 1
    while (i < N) and (Ti ≠ X) do
        i ← i + 1
    { i = N or Ti = X}
    if (Ti = X) then
        IX ← i
    else { Ti ≠ X }
        IX ← 0
```

Algoritma Pencarian Sequential Skema Pencarian Dengan Boolean

```
procedure SEQSearchX2 (input T : TabInt, input N : integer, input X : integer,
                       output IX : integer, output Found : boolean)
{ Mencari harga X dalam Tabel T [1..N] secara sekuensial mulai
  dari T1. Hasilnya adalah indeks IX di mana T[IX] = X (i
  terkecil), IX = 0 jika tidak ketemu, dan sebuah boolean Found
  (true jika ketemu) }
```

```
Kamus Lokal
   i : integer { [1..N+1], indeks untuk pencarian }
```

```
ALGORITMA
    Found ← false  { awal pencarian, belum ketemu }
    i ← 1
    while (i <= N) and (not(Found)) do

        if (Ti = X) then
            Found ← true
        else
            i ← i + 1
    { i > N or Found }
    if (Found) then
        IX ← i
    else
        IX ← 0
```

T boleh kosong karena tidak ada pemeriksaan nilai T[i] pada kondisi pengulangan

Algoritma Pencarian Sequential Pada Tabel Terurut

```
procedure SEQSearchSorted (input T : TabInt, input N : integer, input X : integer,
                           output IX : integer)
{ Mencari harga X dalam Tabel T 1..N secara sekuensial mulai
  dari T1. Hasilnya adalah indeks IX di mana TIX = X, IX = 0
  jika tidak ketemu. }
```

```
Kamus Lokal
    i : integer { [1..NMax], indeks untuk pencarian }
```

```
ALGORITMA
    i ← 1
    while (i < N) and (Ti <  X) do
            i ← i + 1
    { i = N or Ti >= X }
    if (Ti = X) then
        IX  ← i
    else  { Ti ≠ X -> Ti > X }
        IX  ← 0
```

Algoritma Pencarian Sequential Dengan Sentinel - Algoritma

```
procedure SEQSearchWithSentinel (input T : TabInt, input N : integer, input X : integer,
                                 output IX : integer)
{ Mencari harga X dalam Tabel T[1..N] secara sekuensial mulai dari T1.
  Hasilnya adalah indeks IX di mana TIX = X (i terkecil), IX = 0 jika tidak ketemu.
  Sentinel diletakkan di TN+1. }
```

```
Kamus Lokal
    i : integer, { [1..N+1] indeks untuk pencarian }
```

```
ALGORITMA
    TN+1 ← X   { pasang sentinel }
    i ← 1
    while (Ti ≠ X) do   { tidak perlu test terhadap batas i, karena pasti berhenti }
         i ← i + 1
    { Ti = X, diperiksa apakah ketemunya di sentinel }
    if (i < N+1) then
       IX  ←  i { ketemu pada elemen tabel }
    else { i = N+1, ditemukan di sentinel, berarti tak ketemu }
       IX  ←  0
```

Counting Sort (Algoritma)

```
procedure CountSORT (input/output T : TabInt, input N : integer)
{ mengurut tabel integer [1..N] dengan pencacahan }

Kamus Lokal
   { ValMin & ValMax: batas Minimum & Maximum nilai di T, hrs diketahui }
   TabCount : array [ValMin..ValMax] of integer;
   i, j : integer; { indeks untuk traversal tabel }
   K : integer; { jml elemen T yg sudah diisi pada pembentukan kembali }

ALGORITMA
if (N > 1) then
        { Inisialisasi TabCount }
        i traversal [ValMin..ValMax]
            TabCount_i ← 0

        { Counting }
        i traversal [1..N]
            TabCount_T[i] ← TabCount_T[i] + 1

        { Pengisian kembali : T_1 ≤ T_2 ≤ ... ≤ T_N }
        K ← 0
        i traversal [ValMin..ValMax]
            if (TabCount_i ≠ 0) then
                j traversal [1..TabCount_i]
                    K ← K + 1
                    T_K ← i
```

Elemen Tabel TabCount diinisialisasi 0

Telusuri TabInt, sambil mengupdate elemen TabCount → TabCount berisi jumlah kemunculan elemen pada TabInt

Telusuri TabCount, untuk mengisi TabInt sesuai isi TabCount → TabInt terurut

Selection Sort (Algoritma)

```
procedure MAXSORT (input/output T : TabInt, input N : integer)
{ mengurut tabel integer [1..N] terurut mengecil dgn maksimum suksesif }

Kamus Lokal
    i : integer        { indeks untuk traversal tabel }
    Pass : integer      { tahapan pengurutan }
    Temp : integer      { memorisasi harga untuk penukaran }
    IMax : integer      { indeks, di mana T[Pass..N] bernilai maksimum }

Algoritma
   if (N > 1) then
        Pass traversal [1..N-1]
              { Tentukan Maximum [Pass..N] }
              IMax ← Pass
              i traversal [Pass+1.. N]
                     if (T_IMax < T_i ) then
                            IMax ←  i
              { T_IMax adalah maximum T[Pass..N] }
              {Tukar T_IMax dengan T_Pass }
              Temp    ← T_IMax
              T_IMax ← T_Pass
              T_Pass ← Temp
              { T_1..Pass terurut: T_1 ≥ T_2 ≥ ... ≥ T_Pass }
   { Seluruh tabel terurut, T_1 ≥ T_2 ≥ ... ≥ T_N }
```

Cari indeks dgn nilai maksimum (di bagian tabel yang belum terurut)

Tukarkan elemen pada indeks maksimum dengan elemen terujung dari bagian tabel yang belum terurut

Insertion Sort (Algoritma)

```
procedure InsertionSORT (input/output T : TabInt, input N : integer)
{ mengurut tabel integer [1..N] dengan insertion }
Kamus Lokal
   i : integer     { indeks untuk traversal tabel }
   Pass : integer  { tahapan pengurutan }
   Temp : integer  { penampung nilai sementara, untuk pergeseran }
ALGORITMA

   if N > 1 then
    { T1 adalah terurut}
       Pass traversal [2..N]

          Temp ← TPass    { Simpan harga T[Pass] sebelum pergeseran }
          { Sisipkan elemen ke Pass dalam T[1..Pass-1] sambil menggeser:}
          i ← Pass-1
          while (Temp < Ti) and (i > 1) do

               Ti+1 ← Ti     { Geser}
               i ← i – 1          { Berikutnya }

          { Temp >= Ti (tempat yg tepat) or i = 1 (sisipkan sbg elmt pertama) }
          if (Temp >= Ti) then
               Ti+1 ← Temp      { Menemukan tempat yg tepat }
          else

               Ti+1 ← Ti
               Ti ← Temp       { sisipkan sbg elemen pertama }
          { T1..Pass terurut membesar: T1≤T2≤T3≤...≤TPass }

      { Seluruh tabel terurut, karena Pass = N: T1≤T2≤...≤TN }
```

Search posisi yang tepat untuk menyisipkan nilai

Bubble Sort (versi optimum) - Algoritma

```
procedure BubleSortPlus (input/output T : TabInt, input N : integer)
{ Mengurut tabel integer [1..N] dengan bubble sort }
{ Pengurutan dihentikan jika tak ada pertukaran  lagi }
KAMUS LOKAL
    i : integer        { indeks untuk traversal tabel }
    Pass : integer     { tahapan pengurutan }
    Temp : integer     { memorisasi untuk pertukaran harga }
    Tukar : boolean    { true jika dalam satu pass ada pertukaran}
ALGORITMA
    if N > 1 then
        Pass ←1
        Tukar ← true    { masih harus ada pertukaran }
        while (Pass ≤ N-1) and (Tukar) do
            Tukar ← false
            K traversal [N..Pass+1]
                if (T_K < T_K-1) then
                    Temp ← T_K
                    T_K ← T_K-1
                    T_K-1 ← Temp
                    Tukar ← true
                { T[1..Pass] terurut: T_1 ≤ T_2 ≤ T_3 ≤ ... ≤ T_Pass }
            { Tukar = true jika ada pertukaran }
            Pass ← Pass + 1    { ke pass yang berikutnya }
        { Seluruh tabel terurut, karena Pass = N: T_1 ≤ T_2 ≤ T_3 ≤ ... ≤ T_N }
```

Abstract Data Type (ADT)

Abstract Data Type (ADT)

ADT adalah pemodelan suatu tipe data yang didefinisikan perilakunya berdasarkan:

data yang terkandung di dalamnya,

himpunan nilai yang mungkin dimiliki oleh data tersebut, serta

operasi yang dapat diterapkan terhadap data tersebut.

Implementasi dari suatu ADT mencakup **struktur data** untuk data yang didefinisikan oleh ADT dan **algoritma** untuk setiap operasi terhadap data tersebut.

ADT, struktur data, dan algoritma adalah salah satu mekanisme modularitas (→ *reusability*) dalam paradigma prosedural.

Ringkasan: ADT

Membuat suatu ADT mencakup:

1) Spesifikasi:

- **Definisi** tipe data
- **Himpunan nilai** yang mungkin
- Daftar **operasi-operasi** (primitif maupun penunjang)
  - **Prekondisi** yang harus terpenuhi sebelum operasi dimulai
  - **State** setelah operasi selesai

2) Implementasi:

- **Struktur data** konkret
- **Implementasi algoritma** setiap operasi
- "**Test case**" untuk setiap operasi pada setiap rentang kemungkinan nilai

Jenis struktur data umum

record atau ***tuple***	***array***	***node-based*** atau ***linked*** (struktur berkait)
• Agregasi beberapa nilai dengan jumlah yang tetap. ▪ Setiap nilai dapat berbeda tipe. ▪ Elemennya diakses dari namanya.	• Sejumlah elemen yang diletakkan di memori secara kontigu. • Setiap elemen bertipe data sama. • Elemennya diakses melalui indeks.	• Sejumlah elemen yang saling terhubung melalui *pointer.* • *S*etiap elemen bertipe data sama. • Elemen diacu oleh elemen sebelumnya.

Contoh spesifikasi ADT dengan notasi algoritmik

Struktur data

```
{ Modul ADT Time }
type Time: <  hours: integer[0..23],
            minutes: integer[0..59],
            seconds: integer[0..59]  >
```

Possible values

```
{ 0 ≤  hours  ≤ 23 }
{ 0 ≤ minutes ≤ 59 }
{ 0 ≤ seconds ≤ 59 }
```

Deklarasi operasi

```
{ Konstruktor: membentuk Time dari komponen-komponennya: h sebagai hours, m sebagai
               minutes, dan s sebagai seconds. }
procedure CreateTime(output T: Time, input h: integer[0..23],
                     input m: integer[0..59], input s: integer[0..59])
{ Mendapatkan komponen hours dari T }
function getHours(T: Time) → integer[0..23]
{ Mendapatkan komponen minutes dari T }
function getMinutes(T: Time) → integer[0..59]
{ Mendapatkan komponen seconds dari T }
function getSeconds(T: Time) → integer[0..59]
{ Selisih antara dua Time, dalam satuan detik }
function difference(start: Time, end: Time) → integer
```

Contoh realisasi operasi dalam notasi algoritmik

Signature operasi

Prasyarat operasi

Ekspektasi hasil operasi pada setiap rentang kemungkinan masukan *(test case)*

```
d: Time) → integer
ime start dan end, dengan syarat start ≤ end. }
```

```
{ EXPECT
      CreateTime(start, 1,2,3); CreateTime(end, 2,3,4)
          ⇒ difference(start,end) = 3661
      CreateTime(start, 2,3,4); CreateTime(end, 2,3,4)
          ⇒ difference(start,end) = 0
      CreateTime(start, 2,3,4); CreateTime(end, 1,2,3)
          ⇒ difference(start,end) = tak terdefinisi }
```

Body operasi

```
KAMUS LOKAL
    startSec, endSec: integer
ALGORITMA
    startSec ← getHours(start)*60*60 + getMinutes(start)*60 + getSeconds(start)
    endSec   ← getHours(end)*60*60   + getMinutes(end)*60   + getSeconds(end)
    → endSec-startSec
```

ADT yang telah dipelajari pada IF1210 AP1

- ADT List
  - ADT List Array Kontigu
  - ADT List Array Tersebar
  - ADT List Array Dinamik
- ADT Matriks
- ADT Queue
- ADT Stack
- ADT Set
- ADT Map (+ Hash Table)
- ADT List Berkait (dengan variasinya)
- ADT Stack dan ADT Queue dengan Representasi Berkait